

Foundations of AI – Applying Fundamental Mathematics in AI Problem Solving Report

Alvyn Shibu - 40356093

1. Introduction

This project applies foundational mathematical methods to the Communities and Crime dataset from the UCI Machine Learning Repository. The dataset contains 1990 US Census socioeconomic factors with FBI Uniform Crime Reports, providing demographic, income, housing and policing statistics for 1994 US communities.

The project's aim is to identify latent socioeconomic factors that influence violent crime rates and to reduce the dataset's dimensionality using Singular Value Decomposition (SVD).

2. Methodology

2.1. Data Preprocessing

2.1.1. Data Quality

The raw data suffered from a large quantity of missing values in policing features and strong multicollinearity among socioeconomic measures. Preprocessing eliminated the missing features and standardised the values, creating a clean numeric matrix suitable for decomposition. The data column names also had to be dynamically loaded into the corresponding features.

2.1.2. Data loading and target isolation.

The dataset was imported and separated into feature matrix x and target variable y as shown in figures 1 and 2.

```
def get_column_names(names_file):
    column_names = []
    with open(names_file, 'r') as file:
        for line in file:
            line = line.strip()
            if line.startswith('@attribute'):
                parts = line.split()
                if len(parts) > 1:
                    name = parts[1]
                    column_names.append(name)
    return column_names

names_file = '../fai-project/communities.names'
column_names = get_column_names(names_file)

df = pd.read_csv("../fai-project/communities.data", sep = ",", names=column_names)
df.head()
```

Figure 1

```
# Separate features and target variable
x = df.drop("ViolentCrimesPerPop", axis=1)
y = df["ViolentCrimesPerPop"]
```

Figure 2

2.1.3. Removal of categorical identifiers.

Non-numeric attributes were dropped as they are identifiers rather than measurable attributes of the community and were only used as location labels. Categorical variables were not encoded since one hot encoding would create

thousands of sparse dummy columns with no quantitative meaning, distort the variance structure needed for SVD, and inflate dimensionality without improving prediction (Atkinson, 2024).

2.1.4. Handling of missing data.

Columns with >84% missing data were dropped, and remaining rows with missing entries were removed.

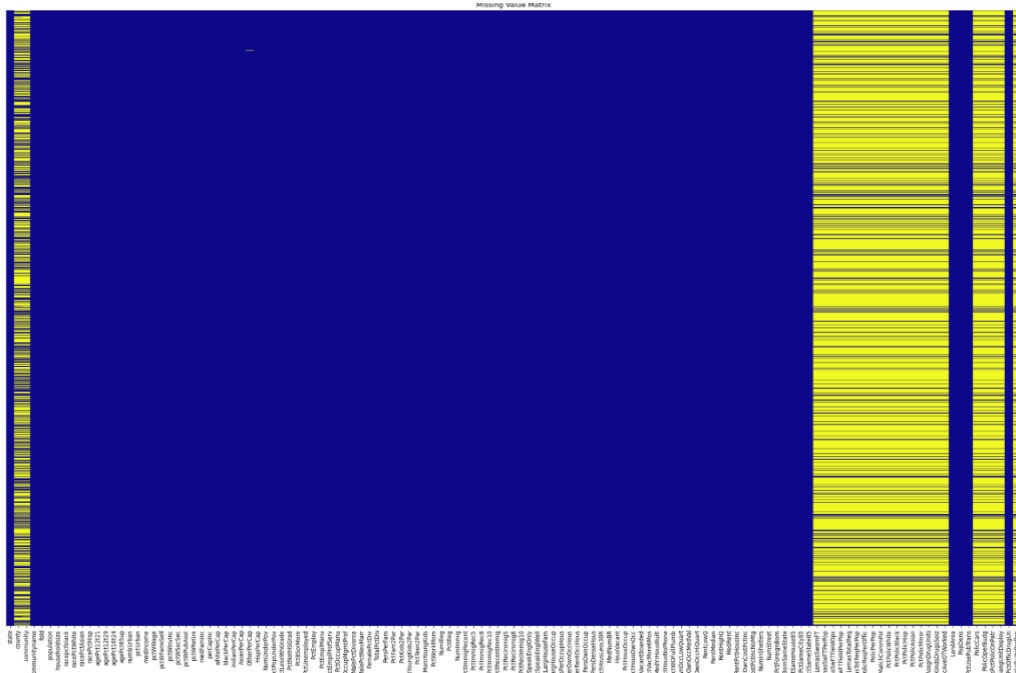


Figure 3

2.1.5. Standardisation

Although the data is already scaled, the data was centred to zero mean and unit variance using `StandardScaler()` which is required for SVD to work correctly as this method assumes that each feature is mean centred so that the resulting components capture variance rather than absolute magnitude. This also prevented features with large values from dominating low range features.

2.1.6. Data Preparation Outcomes

- Original shape: 1994 x 128
- Removed (Categorical + high missing values): 28
- Removed (Row with missing values): 1
- Final: shape after cleaning: 1993 x 100

2.2. Singular Value Decomposition (SVD)

SVD was suitable for this project due to the high dimensionality and multicollinearity of the dataset. SVD decomposes the data into orthogonal bases where each column of V is a linear combination of original features that is orthogonal to the others. This ensures that the analysis operates on independent variables, eliminating the effects of collinearity without discarding variance information. The standardised matrix was decomposed as $X = USV^T$. The cumulative variance curve was used to identify the minimum k capturing 95% of

the total variance. 35 components retained explained 95% of total variance. The data was reduced to 35 dimensions, yielding a compact orthogonal feature space used for interpretation.

3. Exploratory Data Analysis

3.1. Target Variable Distribution and Outliers

When the distribution of violent crime rates is examined (see Figure 4), a significant right skew can be seen. Most communities show relatively low crime rates (between 0.0 and 0.3), with a long tail extending to 1.0. This confirms that areas with high crime rates represent a minority of neighbourhoods and behave as statistical outliers.

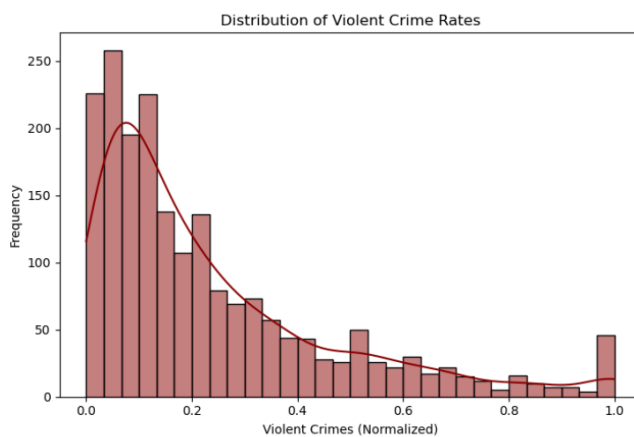


Figure 4

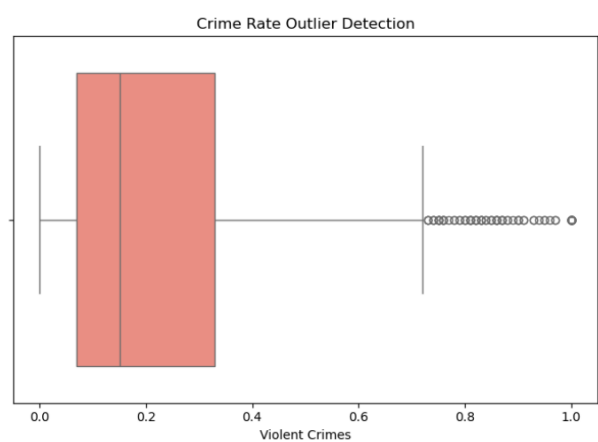


Figure 5

The boxplot corroborates this (see Figure 5): multiple high value points sit beyond the upper whisker, highlighting that the dataset includes some extreme cases. These outliers are real observations, not just random noise, and so must remain in the dataset as they carry important information about high-risk areas.

Summary statistics further confirm this structural skew. The median crime level is significantly lower than the upper quartile, and the extreme maximum values pull the distribution upwards. From a modelling perspective, this asymmetry suggests that:

- High crime areas will have a disproportionate impact on squared error loss.
- Learning algorithms need to account for both the dense cluster of low crime areas and the sparse high crime tail.
- Linear models might not adequately represent the rare high crime instances unless regularisation or nonlinear adjustments are applied.

3.2. Distributional Properties and Normality Testing

To formally assess the normality of the distribution, a Shapiro-Wilk test was conducted (see Figure 7). The statistic (0.8216) and its corresponding p-value of 1.7167×10^{-42} indicate that the null hypothesis can be rejected at any standard significance level. This clarifies that the violent crime variable does not follow a normal distribution.

```
# Normality Test (Shapiro-Wilk)
stat, p_val_norm = st.shapiro(df_svd['ViolentCrimesPerPop'])
print(f"\nNormality Test (Shapiro-Wilk): Statistic={stat:.4f}, P-Value={p_val_norm:.4e}")
if p_val_norm < 0.05:
    print("Insight: The target variable is NOT normally distributed (Reject H0).")
else:
    print("Insight: The target variable looks normally distributed.")
```

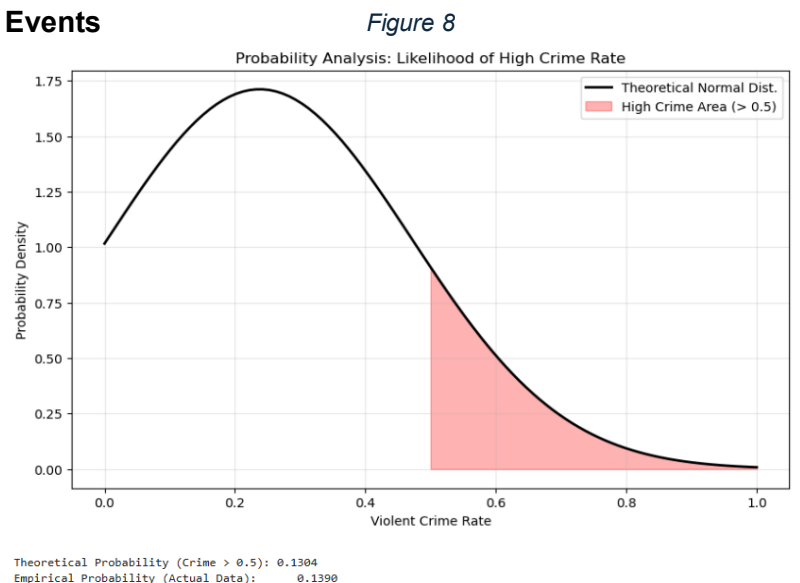
Normality Test (Shapiro-Wilk): Statistic=0.8216, P-Value=1.7167e-42
Insight: The target variable is NOT normally distributed (Reject H0).

Figure 7

This aligns with figure 4 and suggests that parametric modelling assumptions relying on normality of the target must be carefully evaluated during model diagnostics. Non-normality also plays a role in probability modelling as tail probabilities can differ from what a Gaussian approximation would suggest.

3.3. Probability of High Crime Events

Figure 8 displays a fitted normal distribution with a shaded area that shows the probability of crime rates exceeding 0.5. The theoretical probability calculated from the Gaussian approximation is 0.1304, while the actual proportion found in the data is 0.1390.



The close match between theoretical and empirical probabilities indicate that the mean and variance of the distribution provide valuable insights, even when there's a strong skew. However, as the left tail is compressed and the right tail is extended, high crime probabilities tend to be systematically underestimated by a symmetric model.

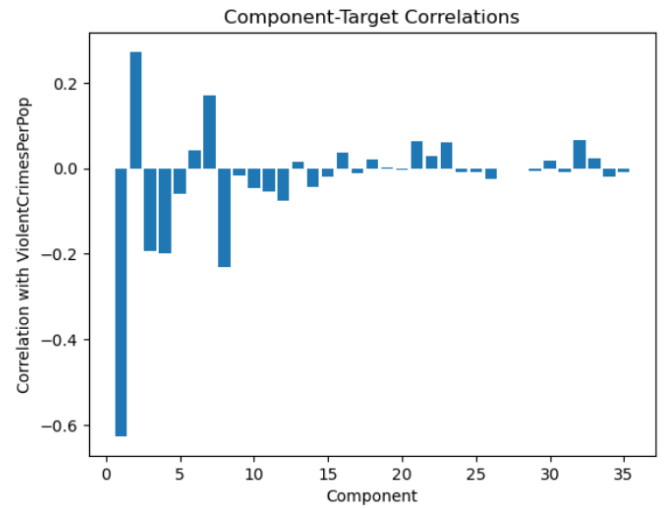
3.4. Component Target Relations

Only a few components show significant correlation with the target as seen in figure 9, confirming that the signals related to crime are concentrated in specific directions of variance.

Key findings:

- PC1: Strong negative correlation with crime. Communities that score high on PC1 have much lower crime rates.
- PC2: Moderate positive correlation pointing to a distinct socioeconomic axis linked to higher crime rates.
- PC3 & PC4: Weaker correlations suggesting that relationship with crime is less

pronounced.



Figure

3.5. Component-Crime Scatter Analysis

Scatterplots (figure 11) with fitted regression lines for PC1-PC4 against violent crime offer a more detailed look at these relationships.

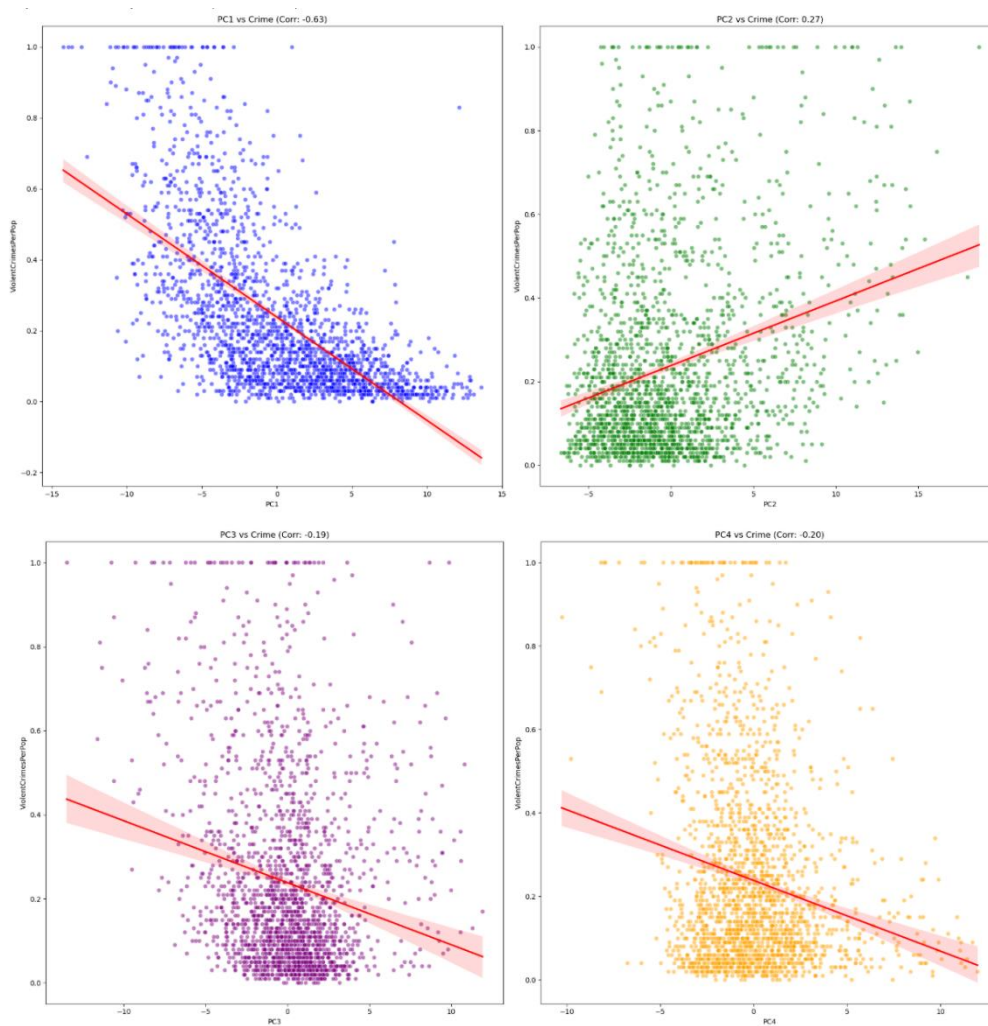


Figure 11

Interpretation:

- PC1: Shows a clear downward trend with low variance dispersion, suggesting a stable linear relationship.
- PC2: Displays a positive slope but with more dispersion, indicating a nonlinear structure and less predictive accuracy.
- PC3/4: Present mild negative slopes; while overshadowed by noise, some directional patterns are noticeable.

This analysis implies that linear regression might effectively model PC1, while neural networks could better capture the nonlinear patterns found in the weaker components.

3.6. Statistical Comparison of High and Low PC1 Communities

PC1 scores were used to split the dataset into high and low groups relative to the median. A two-sample t-test revealed:

- T-statistic: -26.9073
- P-value: 7.9652×10^{-128}

This incredibly strong evidence against the null hypothesis indicates a significant difference in crime levels between the two groups. Thus, PC1 effectively captures a key socioeconomic dimension that distinctly separates low crime areas from high crime areas.

```
# Hypothesis Testing (Independent T-Test)
# H0: There is no difference in crime rates between High and Low PC1 communities
print("\nHypothesis Testing (T-Test)")
pc1_median = df_svd['PC1'].median()
group_high = df_svd[df_svd['PC1'] > pc1_median]['ViolentCrimesPerPop']
group_low = df_svd[df_svd['PC1'] <= pc1_median]['ViolentCrimesPerPop']

t_stat, p_val = st.ttest_ind(group_high, group_low, equal_var=False)
print(f"T-Statistic: {t_stat:.4f}, P-Value: {p_val:.4e}")
if p_val < 0.05:
    print("Reject Null Hypothesis (Significant difference found)")
else:
    print("Fail to Reject Null Hypothesis")
```

Hypothesis Testing (T-Test)
T-Statistic: -26.9073, P-Value: 7.9652e-128
Reject Null Hypothesis (Significant difference found)

Figure 12

3.7. Summary of Findings

The exploratory analysis shows that the dataset has a target variable that is heavily skewed, along with some notable outliers and a significant low dimensional structure. Crime levels are most closely linked to PC1 and have a moderate correlation with the second one, while the other components provide very little predictive insight. The non normal nature of the target variable and the nonlinear patterns observed in PC2-PC4 suggest that more complex models should also be considered. In summary, the transformed feature space offers stable and interpretable axes that will be useful for effective optimisation and model comparison.

4. Principal Component Analysis

4.1. SVD Variance Summary

After applying SVD, the first 35 components explained 95% of total variance, confirming many redundant features. PCA is obtained from SVD via covariance matrix $X^T X = V \Sigma^2 V^T$. The eigenvalues of the covariance matrix correspond to squared singular values. The explained variance per component (figure 13)

showed a steep initial decline followed by a gradual flattening, forming the elbow shape seen in high dimensional data. This indicates that a small subset of components contributes increasingly marginal information. The cumulative variance curve confirmed that:

- PC1-PC5 captured roughly 70-75% of variance, representing dominant socioeconomic and demographic gradients.
- PC6-PC15 added another 15-20%, modelling finer scale patterns such as housing, age and population density.
- PC16-PC35 accounted for the remaining 5-10%, containing community level variation.

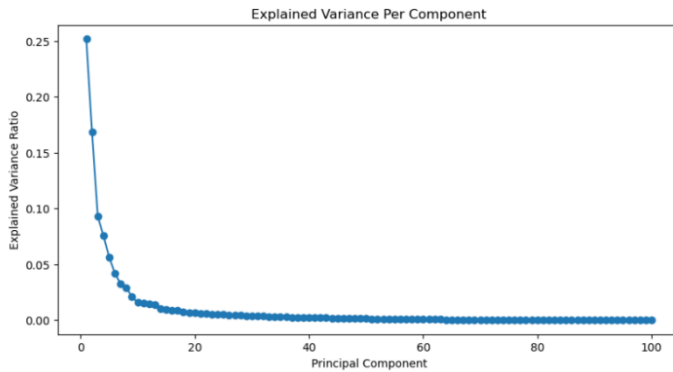


Figure 13

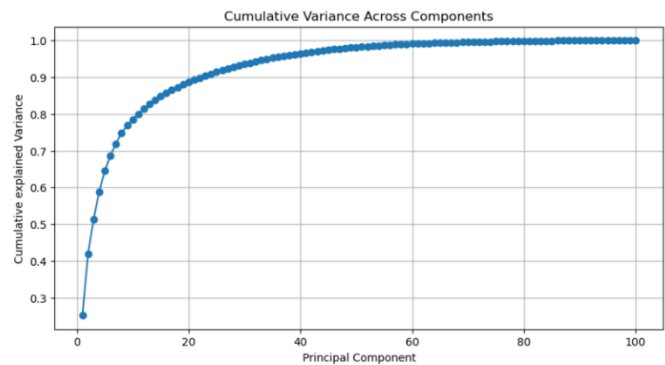


Figure 14

4.2. Feature Loadings

The feature loadings (figure 15) show how strongly each original variable contributes to each principal component. High absolute loadings indicate variables that define the latent socioeconomic dimensions captured by each component. The signs (positive or negative) reflect directional relationships, for example, whether high values of a variable align with affluence or deprivation along that component's axis. Representative patterns for the first components:

- PC1: Economic affluence ↔ deprivation.
- PC2: Immigration and linguistic diversity.
- PC3: Demographic structure.

```
Top contributing features to PC1:
medFamInc      0.182929
medIncome      0.181647
PctKids2Par    0.174699
pctInvInc      0.174152
PctFam2Par     0.171902
PctYoungKids2Par 0.170946
perCapInc      0.169158
PctTeen2Par    0.159731
MedRent        0.149226
whitePerCap    0.147815
Name: PC1, dtype: float64

Most negative contributors:
PctPopUnderPov -0.173255
pctWPubAssst   -0.164001
PctHousNoPhone -0.163531
PctNoH5GGrad   -0.161310
PctUnemployed  -0.158060
PctLess9thGrade -0.145171
PctIllig       -0.144534
PctOccuManu    -0.133915
TotalPctDiv    -0.129078
PctHousLess3BR -0.127764
Name: PC1, dtype: float64

Top contributing features to PC2:
PctRecImmig10  0.219149
PctRecImmig8   0.219093
PctRecImmig5   0.216955
PctRecImmig    0.214450
PctForeignBorn 0.212574
PctNotSpeakEnglWell 0.187471
PctPersDenseHous 0.174681
racePctAsian   0.167428
racePctHispan 0.162173
PctLangHouseFam 0.154788
Name: PC2, dtype: float64

Most negative contributors:
PctSpeakEnglOnly -0.190859
PctBornSameState -0.128161
pctHSecSec       -0.120415
racePctWhite     -0.119453
agePct65Sup      -0.111200
pctWRetire       -0.102740
PctVacMore6Mos   -0.095450
PctWorkMom       -0.085557
PctPersOwnOccup  -0.084104
PctHousOwnOcc    -0.076942
```

Figure 15

4.3. Impact of SVD

- Dimensional simplification: SVD condensed 100 correlated features into 35 orthogonal components, eliminating multicollinearity whilst preserving 95% variance.
- Variance concentration: Although there are 35 components retained, most of the variance is captured in the first few singular directions. This confirms that most socioeconomic indicators describe overlapping aspects of a common deprivation gradient.

- Interpretability: The loadings on the components show clear themes. For example, PC1 shows that a high income, education and family oppose poverty, unemployment and low education. The strong negative correlation with violent crime indicates that economic disadvantage is the principal explanatory factor.

5. Model Optimisation

5.1. Preliminary Learning Rate Investigation

Before formal tuning, both models were tested across a wide range of α values to diagnose stability.

5.1.1. Linear Regression Instability at High Learning Rates

High learning rates ($\alpha > 0.09$) caused the linear regression model to diverge. During these runs the gradients enlarged rapidly and the cost curve shot upward, producing NaN outputs known as gradient explosion. This was expected as the convex quadratic loss surface restricts the maximum stable learning rate.

5.1.2. Neural Network Requiring Larger Learning Rates

Using low α values on the neural network produced almost flat loss curves due to sigmoid induced vanishing gradients. The network only optimised effectively when α was increased substantially (> 0.1) (Arora et al., 2019).

These early results directly informed the ranges used in the formal CV search:

- Linear Regression: $\alpha \in \{0.0001, 0.0005, 0.01, 0.05, 0.07\}$
- Neural Network: $\alpha \in \{0.1, 0.2, 0.5, 0.7, 0.9\}$

This preliminary diagnostic step prevented wasted computation on unstable or ineffective settings.

5.2. Cross Validated Hyperparameter Search

A 5-fold cross-validation process was implemented on the 80% training subset, making sure the test set stayed completely untouched. The mean-fold MSE was used as the criterion for selection.

5.2.1. Linear Regression Results

In figure 16, the learning rate sweep reveals a significant drop in error at $\alpha = 0.005$, while higher values tend to increase the error due to instability.

5.2.2. Neural Network Results

The neural network curve in Figure 17 displays a wide stable region, achieving optimal performance at $\alpha = 0.7$.

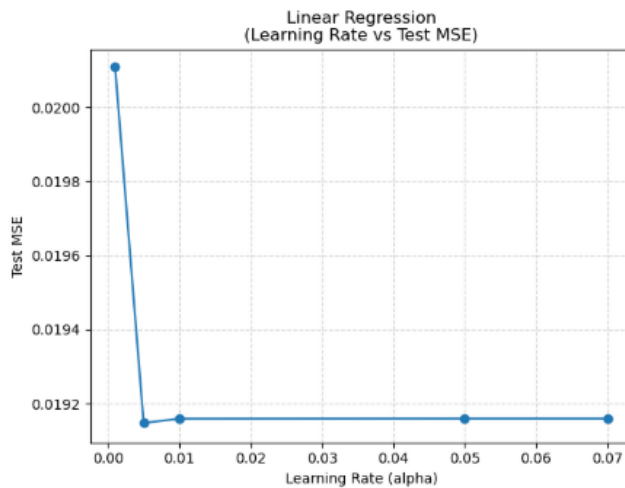


Figure 16

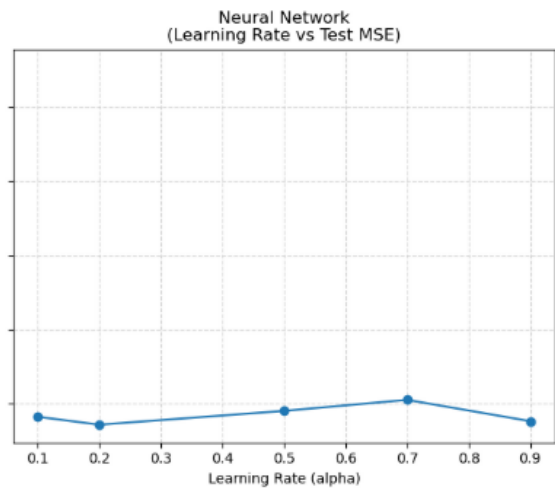


Figure 17

5.3. Final Model Training and Evaluation

With the selected hyperparameters, both models were retrained on the complete training set and evaluated on the untouched test set.

Performance Summary:

Model	Train MSE	Test MSE	Train R2	Test R2
Linear Regression	0.01800	0.02018	0.67004	0.61970
Neural Network	0.01776	0.02019	0.67455	0.61947

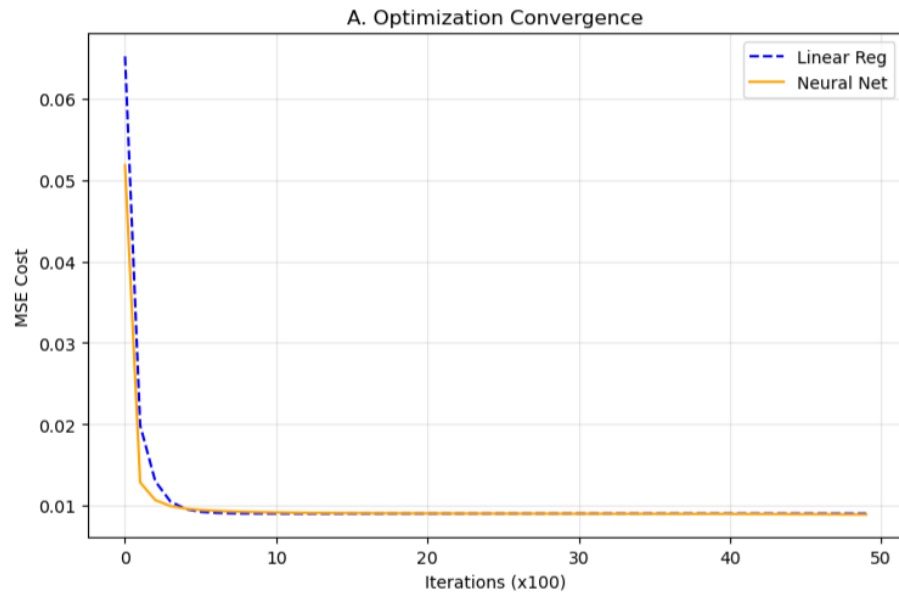
Both models achieve essentially identical generalisation performance with test MSE ≈ 0.020 . This confirms that the PCA/SVD reduced feature space is largely linear and that the target variable contains substantial noise (Braga et al., 2019), consistent with the skewed distribution identified in the EDA.

5.4. Training Dynamics and Diagnostic Visualisations

5.4.1. Convergence Curves

Both models converge rapidly within the first few hundred iterations. The neural network decreases more gradually but stabilises at a slightly lower train MSE as shown below in figure 18.

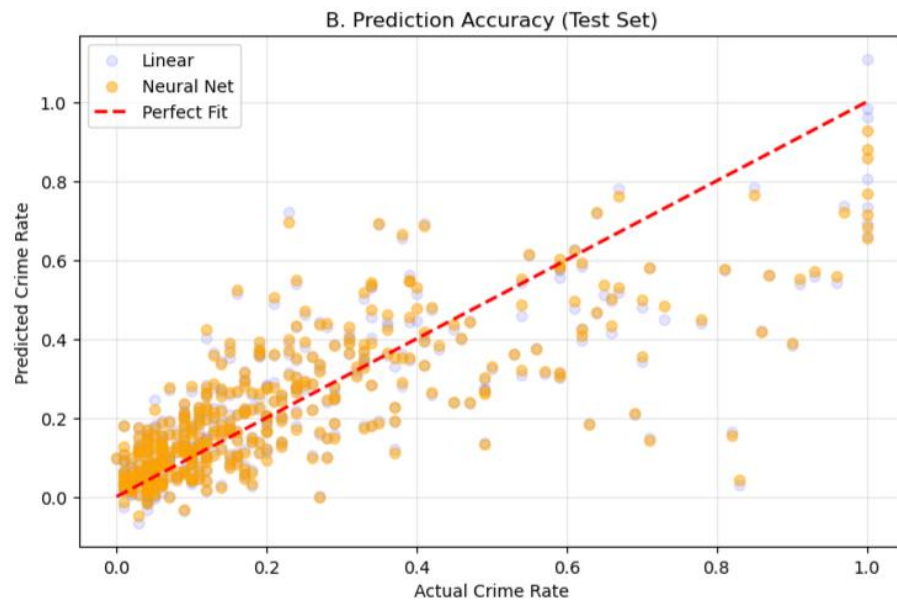
Figure 18



5.4.2. Predicted vs Actual

Both models follow the diagonal trend, though high crime values are systematically underpredicted, consistent with the sparsity and skew in violent crime observations shown in the EDA.

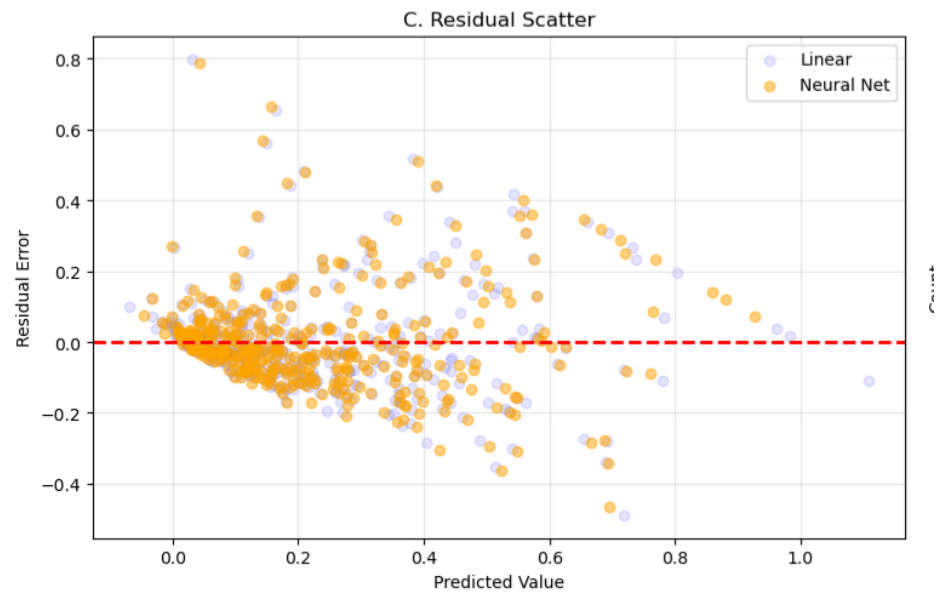
Figure 19



5.4.3. Residual Scatter

Residuals are centred around zero with no structural pattern, indicating correct model specification.

Figure 10



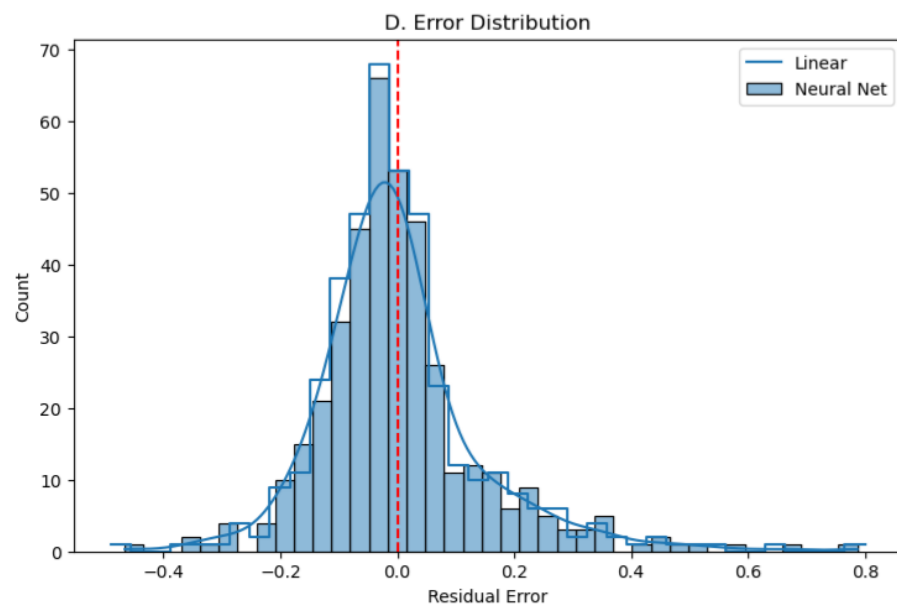
5.4.4. Error Distribution

Residual histograms show:

- Near normal shape
- Slightly unequal variance
- Mildly heavier tails for linear regression

These diagnostics confirm that both models generalise appropriately and exhibit no overfitting.

Figure 21



6. Markov Decision Process

6.1. MDP Formulation

The task is framed as a series of decisions made under uncertainty, where communities move between various crime-risk states, and the policymaker chooses an intervention at each step.

States (S):

There are four distinct states that represent levels of community safety:

- low_risk
- moderate_risk
- high_risk
- critical (terminal)

These states illustrate a growing continuum of crime severity, with the critical state being terminal.

Actions (A):

In each state, there are three intervention options:

- maintain: Keep current resources as they are.
- increase_patrols: Send out more reactive policing.
- invest_prevention: Put money into preventative or community programs.

Each action varies in cost and long-term impact.

6.2. Transition Model

The carefully selected transition probabilities reflect realistic dynamics: investing in prevention lowers the chance of escalation, while inaction in high-risk areas raises the likelihood of slipping into critical.

Examples:

- In low_risk, invest_prevention keeps the community stable with a probability of 0.85.
- In high_risk, maintain produces a 50% chance of immediate collapse to critical.
- In moderate_risk, proactive actions shift probability towards safer states.

6.3. Reward Function

Rewards clarify the desirability of outcomes:

- A positive reward is given for maintaining low_risk conditions.
- Penalties increase as crime severity escalates.

The cost of actions is factored in, for example, increasing patrols in a lower reward in low_risk due to resource expenses. The critical state results in a terminal reward of -50 for all actions, highlighting the severe societal damage.

6.4. Value Iteration

Value iteration computes the optimal value function $V^*(s)$ by applying the Bellman optimality operator (shown below) until convergence.

$$V(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} P(s, a, s') V(s') \right)$$

Hyperparameters:

- Discount factor: $\gamma = 0.9$

- Convergence threshold: $\theta = 1e-4$

All values were initialised at zero. The algorithm iteratively updates each non terminal state until the maximum change across states fell below the convergence threshold.

6.5. Optimal State Values

After running the value iteration, the following optimal state values were

```
Optimal State Values (V*):
State      low_risk: V* = 40.923
State moderate_risk: V* = 37.561
State      high_risk: V* = 29.155
State      critical: V* = 0.000
```

calculated:

Interpretation

- The low_risk state has the highest long-term value as expected.
- moderate_risk still supports substantial long-term value if effective interventions are applied.
- High_risk carries significant long-term cost due to high probability of collapse to critical.
- The critical state's value is zero as it is terminal and produces no further reward.

6.6. Optimal Policy

Using V^* , the greedy optimal policy was extracted:

```
Optimal Policy ( $\pi^*$ ):
State      low_risk: best action = maintain
State moderate_risk: best action = invest_prevention
State      high_risk: best action = invest_prevention
State      critical: best action = None
```

Interpretation:

- low_risk: maintain
 - Maintaining current resources is optimal as the community as already stable, and additional interventions incur unnecessary costs.
- moderate_risk: invest_prevention
 - Prevention yields the highest long-term value, it provides a strong probability of moving to low_risk while generating positive reward.
- high_risk: invest_prevention
 - Increasing patrols reactively reduces escalation but has lower long-term expected value than breaking the escalation cycle through prevention.
- critical: none
 - Terminal state: no decision is available.

6.7. Discussion of Optimal Policy

The best approach focuses on taking preventative measures rather than just reacting to crime, especially in low-risk neighbourhoods where keeping things stable is enough. This is backed by criminological studies that show community based strategies over time are much more effective than simply responding to incidents when it comes to lowering the chances of escalation. The model therefore encodes a key strategic insight: Intervene early and invest in prevention before communities enter high-risk cycles.

7. Discussion, Interpretation & AI Ethical Considerations

7.1. Interpretation of Empirical Results

7.1.1. Data Behaviour and Feature Analysis

The exploratory analysis uncovered three key characteristics that influenced the modelling process:

1. The target variable shows a significant right skew and isn't normally distributed, as confirmed by visual checks and the Shapiro-Wilk test ($p < 0.001$).
2. There are notable outliers, particularly with a long tail of communities with high crime rates.
3. The underlying components reflect the socioeconomic structure related to crime, with the strongest links found in PC1 and PC2.

This suggests that crime levels aren't evenly spread out, and the predictors reveal deeper structural differences rather than just straightforward linear relationships.

7.1.2. Impact of SVD/PCA

Dimensionality reduction helped eliminate redundancy and enhanced computational stability without compromising predictive power. The components that had the strongest correlations highlighted significant structural differences:

PC1: shows a negative correlation with crime rates, indicating communities with better socioeconomic conditions.

PC2: shows a positive correlation, pointing to structural factors associated with higher crime rates.

This transformation reduced noise, multicollinearity and created a more stable optimisation environment for both models.

7.1.3. Model Optimisation and Performance

During the optimisation phase, it became clear that the two models reacted differently to gradient descent:

Linear Regression: Needed a low learning rate. Using higher values led to divergence and exploding gradients.

Neural Network: Required a much larger rate and showed signs of underfitting at lower values.

This distinction justified the separate approaches taken for each model.

7.1.4. MDP Interpretation

The MDP introduced a sequential decision-making aspect that reflects how communities manage crime. The best approach leaned heavily towards investing in prevention rather than just reacting to incidents, especially in both moderate and high-risk situations. This finding aligns with what research shows about reducing crime: taking action early leads to the greatest long-term benefits and the least chance of escalation (Weisburd, 2020). Essentially, the MDP serves as a strategic framework that builds on the predictive model, helping to determine how to use risk information effectively.

7.2. Integrated Insights

Together, the components of the system produce three clear insights:

- Prediction alone is insufficient: communities require sequential planning rather than single-step predictions.
- Structural socioeconomic features dominate crime prediction, even after dimensionality reduction.
- Optimised models converge to similar behaviour, suggesting stability but also implying limited benefit from heavier models in this domain.

These conclusions highlight how modelling decisions shape the final AI system and the risk that incorrect assumptions might carry through to later decision-making stages.

7.3. Ethical Considerations

The system focuses on analysing crime data, categorising communities, and suggesting intervention strategies. However, these areas come with significant ethical concerns. The model was assessed based on four key AI ethical principles.

7.3.1. Transparency

Using dimensionality reduction through SVD creates components that can be hard to interpret in relation to the original features (Vellido, 2019). This can cloud understanding on how the model makes its decisions.

Mitigation strategies:

- Create a mapping table that highlights the main contributors to each component.
- Clearly document the entire modelling process in simple language for those who aren't technical.

7.3.2. Fairness

Crime datasets often reflect socioeconomic and racial biases (Richardson et al., 2019). Even though this dataset is anonymised, some structural features might still indirectly represent protected characteristics. Training linear models and neural networks on such data could unintentionally perpetuate systemic inequalities.

Mitigation strategies:

- Assess fairness metrics, such as group-level residuals across different demographic groups.
- If systematic disparities are found, use constrained optimisation or reweighting.

- Treat the model as a tool for decision support rather than an automated decision-maker.

7.3.3. Privacy

While the data represents aggregated communities, any scenario that involves more detailed predictors could risk breaching privacy.

Mitigation strategies:

- Make sure that aggregation stays above a safe threshold.
- Avoid linking to external demographic datasets unless strict governance measures are in place.

7.3.4. Accountability

The system's recommendations, like whether to invest in prevention or increase patrols could significantly impact community policies. Misunderstandings or overreliance on these recommendations could lead to negative outcomes.

Mitigation strategies:

- Keep decisions under human supervision and maintain audit logs of model recommendations.
- Provide estimates of uncertainty alongside predictions.

7.3.5. Harm Minimisation

Predictive policing systems run the risk of over-policing certain groups (Benbouzid, 2019). The MDP's inclination towards pre-emptive measures could exacerbate this issue.

Mitigation strategies:

- Regular review of intervention outcomes.
- Explicit ban of using models to justify escalation.

7.4. Conclusion

Overall, the predictive modelling and the MDP provide actionable insights into crime-risk dynamics, but responsible deployment requires transparent explanations, fairness monitoring, privacy safeguards, and explicit accountability structures. With these mitigations, the system can be used ethically as part of a broader decision support pipeline rather than as an automated authority.

Word count: 3262 words.

References

- Atkinson, W. (2024). *From multiple correspondence analysis to categorical principal component analysis*. **Quality & Quantity**, 58 (2), 1239-1256.
- Braga, A. A., Welsh, B. C., & Schnell, C. (2019). *Can Policing Reduce Violent Crime? A Systematic Review and Meta-Analysis*. *Aggression and Violent Behaviour*, 46, 149-160.
- Hipp, J. R., & Wickes, R. (2017). *Violence in Urban Neighbourhoods: A Longitudinal Study of Collective Efficacy and Socioeconomic Disadvantage*. **Social Science Research**, 68, 39-53.
- Arora, S., Cohen, N., & Golowich, N. (2019). *A Convergence Analysis of Gradient Descent for Deep Linear Neural Networks*. *International Conference on Learning Representations (ICLR)*.

- Weisburd, D. (2020). *Does Crime Just Move Around the Corner? A Review of the Displacement and Diffusion Literature*. Journal of Quantitative Criminology, 36(3), 459-487.
- Vellido, A. (2019). *The Importance of Interpretability and Visualization in Machine Learning for Applications in Medicine and Health Care*. Neural Computing & Applications, 32(24), 18069-18083.
- Richardson, R., Schultz, J., & Crawford, K. (2019). *Dirty Data, Bad Predictions: The Case of Predictive Policing*. New York University Law Review, 94(1), 1-55.
- Benbouzid, B. (2019). *To Predict and to Manage: Predictive Policing in the United States*. Big Data & Society, 6(1), 1-13.
- <https://archive.ics.uci.edu/dataset/183/communities+and+crime>

Appendix 1. Code of the Solution

```
# Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
import scipy.stats as st
from sklearn.model_selection import train_test_split, KFold
```

```
from sklearn.metrics import mean_squared_error, r2_score
```

```
def get_column_names(names_file):  
    column_names = []  
    with open(names_file, 'r') as file:  
        for line in file:  
            line = line.strip()  
            if line.startswith('@attribute'):  
                parts = line.split()  
                if len(parts) > 1:  
                    name = parts[1]  
                    column_names.append(name)  
    return column_names
```

```
names_file = '../fai-project/communities.names'  
column_names = get_column_names(names_file)
```

```
df = pd.read_csv("../fai-project/communities.data", sep = ",", names=column_names)  
df.head()
```

```
# Print shape of dataset
```

```
df.shape
```

```
# Separate features and target variable
```

```
x = df.drop("ViolentCrimesPerPop", axis=1)
```

```
y = df["ViolentCrimesPerPop"]
```

```
# Print shape of data without target variable to ensure only the one column was dropped
```

```
x.shape
```

```

# Print statistics of dataset
df.describe()

# Check for duplicated values
duplicates = df.duplicated()
print("Number of duplicate rows: ", duplicates.sum())

# Visualise the amount of missing data from each column
# Create a variable to identify missing values
missing = x.eq('?')
plt.figure(figsize=(30, 20))
sns.heatmap(missing, cmap='plasma', cbar=False, yticklabels=False)
plt.title('Missing Value Matrix')
plt.show()

# Check percentage of missing values in columns where values are missing
check_cols = ["county", "community", "LemasSwornFT", "LemasSwFTPerPop",
"LemasSwFTFieldOps", "LemasSwFTFieldPerPop", "LemasTotalReq",
"LemasTotReqPerPop", "PolicReqPerOffic", "PolicPerPop", "RacialMatchCommPol",
"PctPolicWhite", "PctPolicBlack", "PctPolicHisp", "PctPolicAsian", "PctPolicMinor",
"OfficAssgnDrugUnits", "NumKindsDrugsSeiz", "PolicAveOTWorked", "PolicCars",
"PolicOperBudg", "LemasPctPolicOnPatr", "LemasGangUnitDeploy", "PolicBudgPerPop"]
missing_pct = (df[check_cols].replace("?", np.nan).isna().mean() * 100)

print(missing_pct)

corr_matrix =
df.corr(numeric_only=True)["ViolentCrimesPerPop"].sort_values(ascending=False)
plt.figure(figsize=(28, 12))
sns.barplot(x=corr_matrix.index, y=corr_matrix.values)
plt.title("Correlation of Features with ViolentCrimesPerPop")
plt.xlabel("Correlation Coefficient")
plt.ylabel("Feature")
plt.xticks(rotation=90)
plt.tight_layout()

```

```
plt.show()
```

```
# Drop features that either: Have a large amount of missing values, are not measurements  
of community characteristics or could cause data leakage
```

```
x_clean = x.drop(["county", "state", "communityname", "community", "fold",  
"LemasSwornFT", "LemasSwFTPerPop", "LemasSwFTFieldOps",  
"LemasSwFTFieldPerPop", "LemasTotalReq", "LemasTotReqPerPop", "PolicReqPerOffic",  
"PolicPerPop", "RacialMatchCommPol", "PctPolicWhite", "PctPolicBlack", "PctPolicHisp",  
"PctPolicAsian", "PctPolicMinor", "OfficAssgnDrugUnits", "NumKindsDrugsSeiz",  
"PolicAveOTWorked", "PolicCars", "PolicOperBudg", "LemasPctPolicOnPatr",  
"LemasGangUnitDeploy", "PolicBudgPerPop"], axis=1)
```

```
# remove single row with missing value for "OtherPerCap" as visualised in heatmap
```

```
x_cleaned = x_clean[x_clean["OtherPerCap"] != '?']
```

```
# find out index of row which was dropped
```

```
missing_in_x = y.index.difference(x_cleaned.index)
```

```
print("Row in y but not in x: ", missing_in_x)
```

```
# remove corresponding row from target so that there is no shape mismatch
```

```
y_cleaned = y.drop(missing_in_x)
```

```
print(y_cleaned.shape)
```

```
print(x_cleaned.columns)
```

```
# Visualise missing values again to ensure nothing was missed
```

```
missing2 = x_cleaned.eq('?')
```

```
plt.figure(figsize=(15, 10))
```

```
sns.heatmap(missing2, cmap='plasma', cbar=False, yticklabels=False)
```

```
plt.title('Missing Value Matrix')
```

```
plt.show()
```

```
# Use a StandardScaler to center and scale by std to preserve shape of variance distribution
```

```
scaler = StandardScaler()
```

```
x_scaled = scaler.fit_transform(x_cleaned)
```

```

# Apply SVD
U, S, Vt = np.linalg.svd(x_scaled, full_matrices=False)
print(U)
print(S)
print(Vt)

# Calculate explained and cumulative variance
explained_variance = (S ** 2) / np.sum(S ** 2)
cumulative_variance = np.cumsum(explained_variance)

# Plot the explained variance
plt.figure(figsize=(10, 5))
plt.plot(np.arange(1, len(S)+1), explained_variance, marker='o')
plt.xlabel("Principal Component")
plt.ylabel("Explained Variance Ratio")
plt.title("Explained Variance Per Component")
plt.show()

# Plot the cumulative variance
plt.figure(figsize=(10, 5))
plt.plot(np.arange(1, len(S)+1), cumulative_variance, marker='o')
plt.xlabel("Principal Component")
plt.ylabel("Cumulative explained Variance")
plt.title("Cumulative Variance Across Components")
plt.grid(True)
plt.show()

# Select top 'k' largest singular values in  $\Sigma$ 
k = np.argmax(cumulative_variance >= 0.95) + 1
k

print("Explained variance (95%): ", cumulative_variance[k-1])

```

```

# Principal components are the right singular vectors (Vt)
# Reduce data to 'k' dimensions
x_reduced = np.dot(x_scaled, Vt.T[:, :k])
x_reduced

# Output the shape of the reduced data after applying svd
print(x_reduced.shape)

# Output the features that contribute the most and least to each component
contributions = pd.DataFrame(Vt.T[:, :k], index=x_cleaned.columns, columns=[f"PC{i+1}" for
i in range(k)])

for i in range(k):
    print(f"\nTop contributing features to PC{i+1}: ")
    print(contributions.iloc[:, i].sort_values(ascending=False).head(10))
    print("\nMost negative contributors: ")
    print(contributions.iloc[:, i].sort_values().head(10))

# Creates a visual that correlates each component with the target
corrs = [np.corrcoef(x_reduced[:, i], y_cleaned)[0, 1] for i in range(k)]

plt.bar(range(1, k+1), corrs)
plt.xlabel("Component")
plt.ylabel("Correlation with ViolentCrimesPerPop")
plt.title("Component-Target Correlations")
plt.show()

# Create Analysis DataFrame
cols = [f"PC{i+1}" for i in range(x_reduced.shape[1])]
df_svd = pd.DataFrame(x_reduced, columns=cols)
df_svd["ViolentCrimesPerPop"] = y_cleaned.values

```

```

# Examine the distribution and summary statistics
print("Summary Statistics")
print(df_svd['ViolentCrimesPerPop'].describe())

plt.figure(figsize=(14, 5))

# Target Variable Distribution
plt.subplot(1, 2, 1)
sns.histplot(df_svd['ViolentCrimesPerPop'], bins=30, kde=True, color='darkred')
plt.title('Distribution of Violent Crime Rates')
plt.xlabel('Violent Crimes')
plt.ylabel('Frequency')

# Boxplot to check for Outliers in the Target
plt.subplot(1, 2, 2)
sns.boxplot(x=df_svd['ViolentCrimesPerPop'], color='salmon')
plt.title('Crime Rate Outlier Detection')
plt.xlabel('Violent Crimes')

plt.tight_layout()
plt.show()

# Shapiro-Wilk Normality Test
stat, p_val_norm = st.shapiro(df_svd['ViolentCrimesPerPop'])
print(f"\nShapiro-Wilk Normality Test: Statistic={stat:.4f}, P-Value={p_val_norm:.4e}")
if p_val_norm < 0.05:
    print("The target variable is NOT normally distributed (Reject H0).")
else:
    print("The target variable looks normally distributed.")

# Bivariate & Multivariate Analysis (Correlations)
corr_matrix =
df_svd.corr(numeric_only=True)['ViolentCrimesPerPop'].sort_values(ascending=False)

```

```

plt.figure(figsize=(28, 12))
sns.barplot(x=corr_matrix.index, y=corr_matrix.values)
plt.title("Correlation of Features with Components")
plt.xlabel("Components")
plt.ylabel("Feature")
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()

```

```

# Visualizing the linearity between the best predictors and the target
target_corr = df_svd.corrwith(df_svd['ViolentCrimesPerPop']).drop('ViolentCrimesPerPop')
print(f"\nTop Correlated Component: {target_corr.idxmax()} (Corr: {target_corr.max():.4f})")

```

```

fig, axes = plt.subplots(2, 2, figsize=(20, 20))

```

```

# PC1 vs Crime

```

```

sns.scatterplot(x=df_svd['PC1'], y=df_svd['ViolentCrimesPerPop'], alpha=0.5, ax=axes[0,0],
color='blue')

sns.regplot(x=df_svd['PC1'], y=df_svd['ViolentCrimesPerPop'], scatter=False, ax=axes[0,0],
color='red')

axes[0,0].set_title(f'PC1 vs Crime (Corr: {target_corr["PC1"]:.2f})')

```

```

# PC2 vs Crime

```

```

sns.scatterplot(x=df_svd['PC2'], y=df_svd['ViolentCrimesPerPop'], alpha=0.5, ax=axes[0,1],
color='green')

sns.regplot(x=df_svd['PC2'], y=df_svd['ViolentCrimesPerPop'], scatter=False, ax=axes[0,1],
color='red')

axes[0,1].set_title(f'PC2 vs Crime (Corr: {target_corr["PC2"]:.2f})')

```

```

sns.scatterplot(x=df_svd['PC3'], y=df_svd['ViolentCrimesPerPop'], alpha=0.5, ax=axes[1,0],
color='purple')

sns.regplot(x=df_svd['PC3'], y=df_svd['ViolentCrimesPerPop'], scatter=False, ax=axes[1,0],
color='red')

axes[1,0].set_title(f'PC3 vs Crime (Corr: {target_corr["PC3"]:.2f})')

```



```
sns.scatterplot(x=df_svd['PC4'], y=df_svd['ViolentCrimesPerPop'], alpha=0.5, ax=axes[1,1], color='orange')
```

```
sns.regplot(x=df_svd['PC4'], y=df_svd['ViolentCrimesPerPop'], scatter=False, ax=axes[1,1], color='red')
```

```
axes[1,1].set_title(f'PC4 vs Crime (Corr: {target_corr["PC4"]:.2f})')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Hypothesis Testing (Independent T-Test)
```

```
# H0: There is no difference in crime rates between High and Low PC1 communities
```

```
print("\nHypothesis Testing (T-Test)")
```

```
pc1_median = df_svd['PC1'].median()
```

```
group_high = df_svd[df_svd['PC1'] > pc1_median]['ViolentCrimesPerPop']
```

```
group_low = df_svd[df_svd['PC1'] <= pc1_median]['ViolentCrimesPerPop']
```

```
t_stat, p_val = st.ttest_ind(group_high, group_low, equal_var=False)
```

```
print(f"T-Statistic: {t_stat:.4f}, P-Value: {p_val:.4e}")
```

```
if p_val < 0.05:
```

```
    print("Reject Null Hypothesis (Significant difference found)")
```

```
else:
```

```
    print("Fail to Reject Null Hypothesis")
```

```
# Probability Analysis
```

```
# Calculate the probability of Crime Rate > 0.5
```

```
target_mean = df_svd['ViolentCrimesPerPop'].mean()
```

```
target_std = df_svd['ViolentCrimesPerPop'].std()
```

```
threshold = 0.5
```

```
# Visualization
```

```
x_range = np.linspace(df_svd['ViolentCrimesPerPop'].min(),  
df_svd['ViolentCrimesPerPop'].max(), 100)
```

```

pdf_values = st.norm.pdf(x_range, loc=target_mean, scale=target_std)

plt.figure(figsize=(10, 6))
plt.plot(x_range, pdf_values, 'k-', linewidth=2, label='Theoretical Normal Dist.')
x_fill = np.linspace(threshold, x_range.max(), 100)
y_fill = st.norm.pdf(x_fill, loc=target_mean, scale=target_std)
plt.fill_between(x_fill, y_fill, color='red', alpha=0.3, label=f'High Crime Area (> {threshold})')
plt.title('Probability Analysis: Likelihood of High Crime Rate')
plt.xlabel('Violent Crime Rate')
plt.ylabel('Probability Density')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Calculation
prob_theoretical = 1 - st.norm.cdf(threshold, loc=target_mean, scale=target_std)
prob_actual = len(df_svd[df_svd['ViolentCrimesPerPop'] > threshold]) / len(df_svd)

print(f"\nTheoretical Probability (Crime > {threshold}): {prob_theoretical:.4f}")
print(f"Empirical Probability (Actual Data): {prob_actual:.4f}")

#Data preparation
x_data = np.array(x_reduced)
y_data = np.array(y_cleaned).reshape(-1, 1)

# Split Data (80% Train/20% Test)
x_train_full, x_test, y_train_full, y_test = train_test_split(x_data, y_data, test_size=0.2,
random_state=42)

kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Helper functions
def sigmoid(z):
    z = np.clip(z, -500, 500)

```

```
return 1 / (1 + np.exp(-z))
```

```
def sigmoid_derivative(z):
```

```
    s = sigmoid(z)
```

```
    return s * (1 - s)
```

```
def cost_function_mse(y, y_hat):
```

```
    m = len(y)
```

```
    if np.isnan(y_hat).any(): return float('inf')
```

```
    return (1 / (2 * m)) * np.sum((y_hat - y)**2)
```

```
# Model Implementations
```

```
# Linear Regression
```

```
def train_linear_scratch(x, y, alpha=0.01, num_iterations=1000):
```

```
    num_samples, num_features = x.shape
```

```
    np.random.seed(42)
```

```
    W = np.random.randn(num_features, 1) * 0.01
```

```
    b = np.zeros((1, 1))
```

```
    cost_history = []
```

```
# Gradient descent loop
```

```
for i in range(num_iterations):
```

```
    # Forward propagation
```

```
    y_hat = np.dot(x, W) + b
```

```
    # Backpropagation
```

```
    dJ_dz = (y_hat - y)
```

```
    dJ_dW = (1/num_samples) * np.dot(x.T, dJ_dz)
```

```
    dJ_db = (1/num_samples) * np.sum(dJ_dz, axis=0, keepdims=True)
```

```

# Update params
W -= alpha * dJ_dW
b -= alpha * dJ_db

if i % 100 == 0:
    cost_history.append(cost_function_mse(y, y_hat))

return {"W": W, "b": b}, cost_history

def predict_linear(x, params):
    return np.dot(x, params["W"]) + params["b"]

# Neural Network
def train_nn_scratch(x, y, hidden_nodes=5, alpha=0.01, num_iterations=1000):
    num_samples, num_features = x.shape
    np.random.seed(42)

    W1 = np.random.randn(num_features, hidden_nodes) * 0.01
    b1 = np.zeros((1, hidden_nodes))
    W2 = np.random.randn(hidden_nodes, 1) * 0.01
    b2 = np.zeros((1, 1))

    cost_history = []

    for i in range(num_iterations):

        # Forward propagation
        z1 = np.dot(x, W1) + b1
        a1 = sigmoid(z1)
        z2 = np.dot(a1, W2) + b2
        y_hat = z2

```

```

dJ_dz2 = (y_hat - y)
dJ_dW2 = (1/num_samples) * np.dot(a1.T, dJ_dz2)
dJ_db2 = (1/num_samples) * np.sum(dJ_dz2, axis=0, keepdims=True)

dz2_da1 = np.dot(dJ_dz2, W2.T)
dJ_dz1 = dz2_da1 * sigmoid_derivative(z1)
dJ_dW1 = (1/num_samples) * np.dot(x.T, dJ_dz1)
dJ_db1 = (1/num_samples) * np.sum(dJ_dz1, axis=0, keepdims=True)

# Update params
W1 -= alpha * dJ_dW1
b1 -= alpha * dJ_db1
W2 -= alpha * dJ_dW2
b2 -= alpha * dJ_db2

if i % 100 == 0:
    cost_history.append(cost_function_mse(y, y_hat))

return {"W1": W1, "b1": b1, "W2": W2, "b2": b2}, cost_history

def predict_nn(x, params):
    z1 = np.dot(x, params["W1"]) + params["b1"]
    a1 = sigmoid(z1)
    z2 = np.dot(a1, params["W2"]) + params["b2"]
    return z2

# Hyperparameter refinement
print("\nFinding Optimal Parameters")

lr_search_alphas = [0.001, 0.005, 0.01, 0.05, 0.07]
nn_search_alphas = [0.1, 0.2, 0.5, 0.7, 0.9]

```

```
search_nodes = [4, 5, 6]
```

```
lr_tuning_curve = []
```

```
nn_tuning_curve = []
```

```
# Tune LinReg
```

```
print("Tuning Linear Regression")
```

```
best_lr_alpha = None
```

```
best_lr_cv_mse = float('inf')
```

```
lr_tuning_curve = []
```

```
for alpha in lr_search_alphas:
```

```
    fold_errors = []
```

```
    for train_idx, val_idx in kf.split(x_train_full):
```

```
        X_tr = x_train_full[train_idx]
```

```
        y_tr = y_train_full[train_idx]
```

```
        X_val = x_train_full[val_idx]
```

```
        y_val = y_train_full[val_idx]
```

```
        params, _ = train_linear_scratch(X_tr, y_tr, alpha=alpha, num_iterations=2000)
```

```
        preds = predict_linear(X_val, params)
```

```
        if np.isnan(preds).any():
```

```
            fold_errors.append(float('inf'))
```

```
        else:
```

```
            fold_errors.append(mean_squared_error(y_val, preds))
```

```
    avg_mse = np.mean(fold_errors)
```

```
    lr_tuning_curve.append(avg_mse)
```

```
    if avg_mse < best_lr_cv_mse:
```

```

best_lr_cv_mse = avg_mse
best_lr_alpha = alpha

print(f"Best LR Params: Alpha={best_lr_alpha}")

# Tune Neural Network
print("2. Tuning Neural Network")
best_nn_alpha = None
best_nn_alpha_mse = float('inf')
nn_tuning_curve = []

for alpha in nn_search_alphas:
    fold_errors = []

    for train_idx, val_idx in kf.split(x_train_full):
        X_tr = x_train_full[train_idx]
        y_tr = y_train_full[train_idx]
        X_val = x_train_full[val_idx]
        y_val = y_train_full[val_idx]

        params, _ = train_nn_scratch(X_tr, y_tr, hidden_nodes=5, alpha=alpha,
num_iterations=2000)
        preds = predict_nn(X_val, params)

        if np.isnan(preds).any():
            fold_errors.append(float('inf'))
        else:
            fold_errors.append(mean_squared_error(y_val, preds))

    avg_mse = np.mean(fold_errors)
    nn_tuning_curve.append(avg_mse)

```

```

if avg_mse < best_nn_alpha_mse:
    best_nn_alpha_mse = avg_mse
    best_nn_alpha = alpha

best_nn_nodes = None
best_nn_nodes_mse = float('inf')

for h in search_nodes:
    fold_errors = []

    for train_idx, val_idx in kf.split(x_train_full):
        X_tr = x_train_full[train_idx]
        y_tr = y_train_full[train_idx]
        X_val = x_train_full[val_idx]
        y_val = y_train_full[val_idx]

        params, _ = train_nn_scratch(X_tr, y_tr, hidden_nodes=h, alpha=best_nn_alpha,
num_iterations=2000)
        preds = predict_nn(X_val, params)

        if np.isnan(preds).any():
            fold_errors.append(float('inf'))
        else:
            fold_errors.append(mean_squared_error(y_val, preds))

    avg_mse = np.mean(fold_errors)

    if avg_mse < best_nn_nodes_mse:
        best_nn_nodes_mse = avg_mse
        best_nn_nodes = h

```



```
print(f"Best NN Params: Nodes={best_nn_params['nodes']},  
Alpha={best_nn_params['alpha']}")
```

```
# Visualisations
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=True)
```

```
#LinReg subplot
```

```
# Filter for failed runs
```

```
lr_valid = [(a, m) for a, m in zip(lr_search_alphas, lr_tuning_curve) if m is not None]
```

```
if lr_valid: # make sure there's at least one valid point
```

```
    lr_alphas_plot, lr_mse_plot = zip(*lr_valid)
```

```
    axes[0].plot(lr_alphas_plot, lr_mse_plot, marker='o')
```

```
    axes[0].set_title('Linear Regression\n(Learning Rate vs CV MSE)')
```

```
    axes[0].set_xlabel('Learning Rate (alpha)')
```

```
    axes[0].set_ylabel('Test MSE')
```

```
    axes[0].grid(True, linestyle='--', alpha=0.5)
```

```
# Neural Network subplot
```

```
nn_valid = [(a, m) for a, m in zip(nn_search_alphas, nn_tuning_curve) if m is not None]
```

```
if nn_valid:
```

```
    nn_alphas_plot, nn_mse_plot = zip(*nn_valid)
```

```
    axes[1].plot(nn_alphas_plot, nn_mse_plot, marker='o')
```

```
    axes[1].set_title('Neural Network\n(Learning Rate vs CV MSE)')
```

```
    axes[1].set_xlabel('Learning Rate (alpha)')
```

```
    axes[1].grid(True, linestyle='--', alpha=0.5)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Train using full data
```

```
lr_params, lr_cost = train_linear_scratch(  
    x_train_full, y_train_full,
```

```
    alpha=best_lr_alpha,  
    num_iterations=5000  
)
```

```
nn_params, nn_cost = train_nn_scratch(  
    x_train_full, y_train_full,  
    hidden_nodes=best_nn_nodes,  
    alpha=best_nn_alpha,  
    num_iterations=5000  
)
```

```
# Train preds
```

```
lr_train_pred = predict_linear(x_train_full, lr_params)  
nn_train_pred = predict_nn(x_train_full, nn_params)
```

```
# Test preds
```

```
lr_test_pred = predict_linear(x_test, lr_params)  
nn_test_pred = predict_nn(x_test, nn_params)
```

```
# Results
```

```
print(f"\n{'Model':<20} | {'Train MSE':<10} | {'Test MSE':<10} | {'Train R2':<10} | {'Test R2':<10}")  
  
print("-" * 75)
```

```
# Metrics Calculation
```

```
mse_lr_tr = mean_squared_error(y_train_full, lr_train_pred)  
mse_lr_te = mean_squared_error(y_test, lr_test_pred)  
r2_lr_tr = r2_score(y_train_full, lr_train_pred)  
r2_lr_te = r2_score(y_test, lr_test_pred)  
  
print(f"{'Linear Regression':<20} | {mse_lr_tr:.5f} | {mse_lr_te:.5f} | {r2_lr_tr:.5f} |  
{r2_lr_te:.5f}")
```

```

mse_nn_tr = mean_squared_error(y_train_full, nn_train_pred)
mse_nn_te = mean_squared_error(y_test, nn_test_pred)
r2_nn_tr = r2_score(y_train_full, nn_train_pred)
r2_nn_te = r2_score(y_test, nn_test_pred)
print(f"{'Neural Network':<20} | {mse_nn_tr:.5f} | {mse_nn_te:.5f} | {r2_nn_tr:.5f} | {r2_nn_te:.5f}")

```

Visualisations

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
```

Optimization convergence graph

```

axes[0,0].plot(lr_cost, label='Linear Reg', color='blue', linestyle='--')
axes[0,0].plot(nn_cost, label='Neural Net', color='orange')
axes[0,0].set_title('A. Optimization Convergence')
axes[0,0].set_xlabel('Iterations (x100)')
axes[0,0].set_ylabel('MSE Cost')
axes[0,0].legend()
axes[0,0].grid(True, alpha=0.3)

```

Predicted vs Actual graph

```

axes[0,1].scatter(y_test, lr_test_pred, color='blue', alpha=0.1, label='Linear')
axes[0,1].scatter(y_test, nn_test_pred, color='orange', alpha=0.5, label='Neural Net')
axes[0,1].plot([0, 1], [0, 1], 'r--', linewidth=2, label='Perfect Fit')
axes[0,1].set_title('B. Prediction Accuracy (Test Set)')
axes[0,1].set_xlabel('Actual Crime Rate')
axes[0,1].set_ylabel('Predicted Crime Rate')
axes[0,1].legend()
axes[0,1].grid(True, alpha=0.3)

```

Residual Scatter graph

```

res_lr = y_test - lr_test_pred
res_nn = y_test - nn_test_pred
axes[1,0].scatter(lr_test_pred, res_lr, color='blue', alpha=0.1, label='Linear')

```

```
axes[1,0].scatter(nn_test_pred, res_nn, color='orange', alpha=0.5, label='Neural Net')
axes[1,0].axhline(0, color='red', linestyle='--', linewidth=2)
axes[1,0].set_title('C. Residual Scatter')
axes[1,0].set_xlabel('Predicted Value')
axes[1,0].set_ylabel('Residual Error')
axes[1,0].legend()
axes[1,0].grid(True, alpha=0.3)
```

```
# Residual Histogram graph
```

```
sns.histplot(res_lr, color='blue', element="step", fill=False, label='Linear', ax=axes[1,1])
sns.histplot(res_nn, color='orange', kde=True, label='Neural Net', ax=axes[1,1])
axes[1,1].axvline(0, color='red', linestyle='--')
axes[1,1].set_title('D. Error Distribution')
axes[1,1].set_xlabel('Residual Error')
axes[1,1].legend()
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# States represent community crime / safety levels
```

```
states = ['low_risk', 'moderate_risk', 'high_risk', 'critical']
```

```
terminal_state = 'critical'
```

```
# Actions represent possible interventions
```

```
actions = ['maintain', 'increase_patrols', 'invest_prevention']
```

```
transition_prob = {
```

```
    'low_risk': {
```

```
        'maintain': {
```

```
            'low_risk': 0.7,
```

```
            'moderate_risk': 0.3
```

```
    },
    'increase_patrols': {
        'low_risk': 0.8,
        'moderate_risk': 0.2
    },
    'invest_prevention': {
        'low_risk': 0.85,
        'moderate_risk': 0.15
    }
},
'moderate_risk': {
    'maintain': {
        'moderate_risk': 0.6,
        'high_risk': 0.4
    },
    'increase_patrols': {
        'low_risk': 0.4,
        'moderate_risk': 0.5,
        'high_risk': 0.1
    },
    'invest_prevention': {
        'low_risk': 0.5,
        'moderate_risk': 0.4,
        'high_risk': 0.1
    }
},
'high_risk': {
    'maintain': {
        'high_risk': 0.5,
        'critical': 0.5
    },
    'increase_patrols': {
```

```

        'moderate_risk': 0.5,
        'high_risk': 0.4,
        'critical': 0.1
    },
    'invest_prevention': {
        'moderate_risk': 0.6,
        'high_risk': 0.3,
        'critical': 0.1
    }
},
'critical': {
    # Terminal state, once in critical stop modelling transitions
    'maintain': {},
    'increase_patrols': {},
    'invest_prevention': {}
}
}

```

Reward function

```

rewards = {
    'low_risk': {
        'maintain': 5,
        'increase_patrols': 3,
        'invest_prevention': 4
    },
    'moderate_risk': {
        'maintain': -1,
        'increase_patrols': 2,
        'invest_prevention': 3
    },
    'high_risk': {
        'maintain': -5,

```

```
    'increase_patrols': 0,  
    'invest_prevention': 1  
},  
'critical': {  
    'maintain': -50,  
    'increase_patrols': -50,  
    'invest_prevention': -50  
}  
}
```

```
gamma = 0.9
```

```
theta = 1e-4
```

```
V = {s: 0.0 for s in states}
```

```
def value_iteration():
```

```
    iteration = 0
```

```
    while True:
```

```
        delta = 0.0
```

```
        for s in states:
```

```
            if s == terminal_state:
```

```
                continue
```

```
            v_old = V[s]
```

```
            action_values = []
```

```
            for a in actions:
```

```
                total = 0.0
```

```
                for next_s, prob in transition_prob[s][a].items():
```

```
                    total += prob * V[next_s]
```

```
                q_sa = rewards[s][a] + gamma * total
```

```
                action_values.append(q_sa)
```

```
            V[s] = max(action_values)
```

```

        delta = max(delta, abs(v_old - V[s]))
    iteration += 1
    if delta < theta:
        break
    return V

```

```

def extract_policy(V_opt):
    policy = {}
    for s in states:
        if s == terminal_state:
            policy[s] = None
            continue
        action_values = {}
        for a in actions:
            total = 0.0
            for next_s, prob in transition_prob[s][a].items():
                total += prob * V_opt[next_s]
            action_values[a] = rewards[s][a] + gamma * total
        best_a = max(action_values, key=action_values.get)
        policy[s] = best_a
    return policy

```

```

optimal_values = value_iteration()
optimal_policy = extract_policy(optimal_values)

```

```

print("Optimal State Values ( $V^*$ ):")
for s in states:
    print(f"State {s:>13}:  $V^* = \{optimal\_values[s]:6.3f\}")$ 

```

```

print("\nOptimal Policy ( $\pi^*$ ):")
for s in states:

```



```
print(f"State {s:>13}: best action = {optimal_policy[s]}")
```